Python E-Course

Module 10 — Morphological Operations

Detailed Notes

Module Overview

Level: Detection

Duration: ~1 week

Prerequisites: Modules 1-9

What You'll Learn

- Apply erosion, dilation, opening, closing on binary and grayscale images.
- Use morphological gradient, top-hat, black-hat.
- Build structuring elements with cv2.getStructuringElement.
- Clean noisy masks reliably.

Lessons

#	Lesson	Duration
1	Binary Images & Structuring Elements	25 min
2	Erosion and Dilation	30 min
3	Opening and Closing	25 min
4	Gradient, Top-Hat, Black-Hat	25 min
5	Practical Mask Cleaning	25 min

Assessment

- Exercises - Quiz - Assignment - Mini Project

What's Next

Module 11: Segmentation & Thresholding

Lesson 1: Binary Images & Structuring Elements

Module: 10 — Morphological Operations

Duration: 25 minutes

Difficulty: Beginner-Intermediate

Learning Objectives

- Define a binary image and a structuring element (SE / kernel).
- Build SEs of different shapes with cv2.getStructuringElement.
- Understand how the SE shape determines a morphology op's effect.

Prerequisites

- Modules 1-9

1. Why This Matters

Morphology operates on binary masks (output of thresholding or color filtering) to clean them up — remove specks, fill holes, separate touching objects. The structuring element is the morphology equivalent of a convolution kernel — it determines what "a neighbourhood" means.

2. Concept

2.1 Binary image

A (H, W) uint8 image with only two values: 0 (off) and 255 (on). Output of cv2.threshold or cv2.inRange.

2.2 Structuring element (SE)

A small binary shape (3×3, 5×5, etc.) that defines the local neighbourhood examined at each pixel.

2.3 OpenCV API

```
se = cv2.getStructuringElement(shape, ksize, anchor=(-1,-1))
```

Shape constants:

Constant	Shape
cv2.MORPH_RECT	full rectangle of 1s
cv2.MORPH_ELLIPSE	round disc
cv2.MORPH_CROSS	+ shape

Example:

```
se = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (5, 5))
print(se)
```

Output:

```
[[0 0 1 0 0]
 [1 1 1 1 1]
 [1 1 1 1 1]
 [1 1 1 1 1]
 [0 0 1 0 0]]
```

2.4 Shape choice — when it matters

Choice	Use
RECT (square)	Default; fast
ELLIPSE	Most natural for round / curved objects
CROSS	Preserves thin straight lines better

For most denoising work, ELLIPSE looks best. For text / line work, CROSS preserves more structure.

2.5 Size — kernel size matters more than shape

Bigger SE = bigger effect per op. A 9×9 erosion removes much more than a 3×3 erosion. Tune size first, shape second.

2.6 Custom shapes

You can also build SEs manually:

```
import numpy as np
se = np.array([[0, 0, 1, 0, 0],
               [0, 1, 1, 1, 0],
               [1, 1, 1, 1, 1],
               [0, 1, 1, 1, 0],
               [0, 0, 1, 0, 0]], dtype=np.uint8)
```

Useful for unusual shapes (diagonal lines, custom motifs).

3. Code Examples

Example 1: Build SEs

```
import cv2
for s, name in [(cv2.MORPH_RECT, "rect"),
                (cv2.MORPH_ELLIPSE, "ellipse"),
                (cv2.MORPH_CROSS, "cross")]:
    print(name)
    print(cv2.getStructuringElement(s, (5, 5)))
    print()
```

Expected Output: Three 5×5 matrices: solid 1s (rect), round disc (ellipse), + (cross).

Example 2: Visualise SE shapes at different sizes

```
import cv2, numpy as np
import matplotlib.pyplot as plt

shapes = [(cv2.MORPH_RECT, "rect"),
          (cv2.MORPH_ELLIPSE, "ellipse"),
          (cv2.MORPH_CROSS, "cross")]
```

```
sizes = [3, 5, 9, 15]

fig, axes = plt.subplots(len(shapes), len(sizes), figsize=(10, 6))
for i, (s, name) in enumerate(shapes):
    for j, k in enumerate(sizes):
        se = cv2.getStructuringElement(s, (k, k))
        axes[i, j].imshow(se, cmap='gray', vmin=0, vmax=1)
        axes[i, j].set_title(f"{name} {k}x{k}")
        axes[i, j].axis("off")
plt.tight_layout(); plt.show()
```

Expected Output: A 3×4 grid showing each shape at sizes 3, 5, 9, 15.

Example 3: Build a binary image to test on

```
import cv2, numpy as np
from skimage.data import coins

img = coins()
_, binary = cv2.threshold(img, 80, 255, cv2.THRESH_BINARY)
print("unique:", np.unique(binary))
cv2.imwrite("coins_binary.png", binary)
```

Expected Output: unique: [0 255] and a saved binary mask where the coins are white on black background.

4. Parameter Sensitivity

Param	Effect
ksize	bigger = stronger morphological effect
shape	small differences except for very thin / very round features

5. Common Mistakes

Mistake	What Happens	Fix
Apply morphology to non-binary image	Behaves like grayscale morphology — usually unintended	Threshold first
Forget odd kernel size	Anchor offset; weird shifts	Use odd ksize
Default MORPH_RECT for natural objects	Slightly boxy	Try MORPH_ELLIPSE

6. Practice Tasks

- Build a 7×7 ELLIPSE and print it.
- Build a 9×9 CROSS and print it.
- Threshold coins() at 80 to make a binary image. Save.

7. Key Takeaways

- Morphology = local neighbourhood operations on binary (or grayscale) images.
- Structuring element shape: RECT, ELLIPSE, CROSS.

- Size matters more than shape for most denoising.
- cv2.getStructuringElement(shape, (k, k)) is the standard call.

8. What's Next

The two atomic operations: erosion and dilation.

Lesson 2: Erosion and Dilation

Module: 10 — Morphological Operations

Duration: 30 minutes

Difficulty: Intermediate

Learning Objectives

- Apply cv2.erode and cv2.dilate.
- Predict the visual effect (objects shrink / grow).
- Use iterations to chain multiple passes.

Prerequisites

- Module 10 Lesson 1

1. Why This Matters

Erosion and dilation are the two atomic morphology operations. Every other morphology op (opening, closing, gradient, top-hat) is built from these two.

2. Concept

2.1 Erosion — shrink white regions

For each pixel:

- Output is 255 only if all pixels under the SE are 255 in the input.
- Otherwise, 0.

Effect: white regions shrink; thin white lines disappear; small white specks vanish.

2.2 Dilation — grow white regions

For each pixel:

- Output is 255 if any pixel under the SE is 255.
- Otherwise, 0.

Effect: white regions grow; thin gaps in white fill in; small black holes shrink.

2.3 OpenCV API

```
out = cv2.erode(img,  kernel, iterations=1)
out = cv2.dilate(img, kernel, iterations=1)
```

- kernel is the SE.
- iterations chains the op N times. erode(..., iterations=3) ≈ erode(...) applied 3 times.

2.4 Duality

Erosion of white = dilation of black, and vice versa. If your foreground is black, just swap the operations.

2.5 Pure-NumPy versions (instructive)

```
def erode_np(bin_img, ksize=3):
    pad = ksize // 2
    padded = np.pad(bin_img, pad, mode='constant', constant_values=0)
    out = np.zeros_like(bin_img)
    for y in range(bin_img.shape[0]):
        for x in range(bin_img.shape[1]):
            if padded[y:y+ksize, x:x+ksize].min() == 255:
                out[y, x] = 255
    return out
```

Slow but shows the "all neighbours" rule clearly. OpenCV's version is hundreds of times faster.

2.6 Grayscale morphology

Erosion / dilation also work on grayscale: erosion = minimum over the SE, dilation = maximum. Useful for shadow / highlight extraction (see top-hat in L4).

3. Code Examples

Example 1: Erode and dilate a binary image

```
import cv2, numpy as np
from skimage.data import coins

img = coins()
_, binary = cv2.threshold(img, 80, 255, cv2.THRESH_BINARY)

se = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (5, 5))
eroded  = cv2.erode(binary,  se)
dilated = cv2.dilate(binary, se)

combo = np.hstack([binary, eroded, dilated])
cv2.imwrite("erode_dilate.png", combo)
```

Expected Output: 3 panels — original mask | shrunken coins (some thin connections gone) | enlarged coins (touching ones merge).

Example 2: Iterations

```
import cv2
from skimage.data import coins

_, binary = cv2.threshold(coins(), 80, 255, cv2.THRESH_BINARY)
se = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (3, 3))
for it in [1, 3, 5]:
    cv2.imwrite(f"erode_it{it}.png", cv2.erode(binary, se, iterations=it))
```

Expected Output: Iteration 1: barely shrunken. Iteration 5: coins much smaller; small ones gone.

Example 3: Pure-NumPy verify

```
import cv2, numpy as np

def erode_np(bin_img, ksize=3):
    pad = ksize // 2
    padded = np.pad(bin_img, pad, mode='constant', constant_values=0)
    out = np.zeros_like(bin_img)
    for y in range(bin_img.shape[0]):
        for x in range(bin_img.shape[1]):
            if padded[y:y+ksize, x:x+ksize].min() == 255:
                out[y, x] = 255
    return out

img = np.zeros((20, 20), dtype=np.uint8)
img[5:15, 5:15] = 255

mine = erode_np(img, 3)
cv = cv2.erode(img, cv2.getStructuringElement(cv2.MORPH_RECT, (3, 3)))
print("equal?", np.array_equal(mine, cv))
```

Expected Output: equal? True

Example 4: Grayscale dilation = local max

```
import cv2
from skimage.data import camera

se = cv2.getStructuringElement(cv2.MORPH_RECT, (7, 7))
dilated = cv2.dilate(camera(), se)
cv2.imwrite("camera_dilate_gray.png", dilated)
```

Expected Output: Cameraman image becomes brighter, fine dark detail eroded; bright spots grow.

4. Parameter Sensitivity

Param	Effect
ksize	bigger = stronger shrink / grow per pass
iterations	linear stacking; iter=3 with k=3 ≈ k=7 once
SE shape	round / cross / rect — minor differences

5. Common Mistakes

Mistake	What Happens	Fix
Erode white when foreground is black	Wrong direction	Invert with cv2.bitwise_not first, or use dilate
Forgetting iterations exists	Need huge kernel for big effect	Use moderate kernel + iterations
Applying to grayscale by mistake	Min/max instead of binary	Threshold first if you wanted binary

6. Practice Tasks

- Threshold coins() and erode with 5×5 ellipse SE.
- Dilate the same with the same SE; compare.
- Try iterations=3; quantify how many coins remain.

7. Key Takeaways

- Erode: shrink whites; output = min over SE.
- Dilate: grow whites; output = max over SE.
- iterations stacks the effect.
- Together they form the foundation for all morphology.

8. What's Next

Opening and closing — the "denoising" combos.

Lesson 3: Opening and Closing

Module: 10 — Morphological Operations

Duration: 25 minutes

Difficulty: Intermediate

Learning Objectives

- Apply MORPH_OPEN and MORPH_CLOSE via cv2.morphologyEx.
- Remember the "key in lock" mnemonic.
- Pick opening vs closing for a given mask defect.

Prerequisites

- Module 10 Lesson 2

1. Why This Matters

Erosion and dilation alone change object size. Opening and closing clean masks while preserving size — they're what you reach for to remove specks (opening) or fill small holes (closing).

2. Concept

2.1 Definitions

- Opening = erosion followed by dilation, same SE.
- Closing = dilation followed by erosion, same SE.

2.2 Visual effects

Operation	Removes	Preserves
Opening	small white specks; thin protrusions	overall object size
Closing	small black holes inside white; thin gaps	overall object size

2.3 The "key in lock" analogy

A structuring element is a "key". For opening: it must fit inside every preserved white region — specks too small for the key are dropped. For closing: it must fit inside every preserved black region — holes too small for the key are filled.

So pick SE size based on the smallest features you want to KEEP.

2.4 OpenCV API

```
cv2.morphologyEx(img, cv2.MORPH_OPEN,  se)
cv2.morphologyEx(img, cv2.MORPH_CLOSE, se)
```

Equivalent manually:

```
opened = cv2.dilate(cv2.erode(img,  se), se)
closed = cv2.erode (cv2.dilate(img, se), se)
```

2.5 Common pipeline

For a noisy mask:

```
mask = cv2.morphologyEx(mask, cv2.MORPH_OPEN,  small_se)   # despeckle
mask = cv2.morphologyEx(mask, cv2.MORPH_CLOSE, small_se)   # fill small holes
```

2.6 Confusion check

If you swap them and apply CLOSE first then OPEN, you might over-fill then re-create the same problem you opened to remove. Order matters.

3. Code Examples

Example 1: Despeckle with opening

```
import cv2, numpy as np

np.random.seed(0)
mask = (np.random.rand(200, 200) < 0.15).astype(np.uint8) * 255
cv2.imwrite("noisy_mask.png", mask)

se = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (5, 5))
opened = cv2.morphologyEx(mask, cv2.MORPH_OPEN, se)
cv2.imwrite("opened.png", opened)
print("speckles before:", (mask > 0).sum(), "after:", (opened > 0).sum())
```

Expected Output: Far fewer white pixels after opening — small specks gone.

Example 2: Fill holes with closing

```
import cv2, numpy as np

# Build a square with little holes in it
mask = np.zeros((200, 200), dtype=np.uint8)
mask[50:150, 50:150] = 255
np.random.seed(0)
holes = np.random.rand(*mask.shape) < 0.05
mask[holes] = 0
cv2.imwrite("holed_mask.png", mask)

se = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (5, 5))
closed = cv2.morphologyEx(mask, cv2.MORPH_CLOSE, se)
cv2.imwrite("closed.png", closed)
```

Expected Output: The square has its small black holes filled in, while remaining a square (closing preserves overall size).

Example 3: OPEN-then-CLOSE pipeline

```
import cv2, numpy as np
from skimage.data import coins
```

```
_, mask = cv2.threshold(coins(), 80, 255, cv2.THRESH_BINARY)
se = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (5, 5))
mask = cv2.morphologyEx(mask, cv2.MORPH_OPEN,  se)
mask = cv2.morphologyEx(mask, cv2.MORPH_CLOSE, se)
cv2.imwrite("coins_clean.png", mask)
```

Expected Output: Coin mask with edges smoothed and any small specks / holes cleaned up.

Example 4: SE size determines what survives

```
import cv2, numpy as np

mask = np.zeros((200, 200), dtype=np.uint8)
cv2.circle(mask, (50, 100),  10, 255, -1)   # small disc
cv2.circle(mask, (150, 100), 30, 255, -1)   # big disc

for k in [5, 15, 25]:
    se = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (k, k))
    cv2.imwrite(f"open_k{k}.png", cv2.morphologyEx(mask, cv2.MORPH_OPEN, se))
```

Expected Output: k=5 keeps both. k=15 drops the small disc. k=25 drops both. Demonstrates "key in lock" sizing.

4. Parameter Sensitivity

Param	Effect
SE size	larger = more aggressive removal / filling
SE shape	minor differences
iterations	stack the effect

5. Common Mistakes

Mistake	What Happens	Fix
Open then close when close then open was right	Different outcomes	Think about what you're removing vs filling
SE too small	No visible effect	Try a bigger SE
SE too large	Wipes out objects entirely	Match SE to smallest feature you want to keep

6. Practice Tasks

- Build a noisy 15%-speckle mask. Apply opening with 5×5 ellipse.
- Build a holed-square mask. Apply closing.
- Run open-then-close on the binarised coins() and compare to raw threshold.

7. Key Takeaways

- Opening = erode + dilate → removes specks.
- Closing = dilate + erode → fills small holes.
- SE size determines what's preserved.
- Standard mask cleanup: open then close.

8. What's Next

Gradient, top-hat, black-hat — the niche but valuable morphology ops.

Lesson 4: Gradient, Top-Hat, Black-Hat

Module: 10 — Morphological Operations

Duration: 25 minutes

Difficulty: Intermediate

Learning Objectives

- Compute morphological gradient (boundary extraction).
- Use top-hat to find bright details on dark background.
- Use black-hat to find dark details on bright background.

Prerequisites

- Module 10 Lesson 3

1. Why This Matters

These three "compound" ops are the workhorse for boundary outlining (gradient) and removing uneven lighting (top-hat / black-hat). Top-hat in particular is used heavily in OCR pre-processing.

2. Concept

2.1 Morphological gradient

```
gradient = dilation − erosion
```

The thin outline of objects in a binary or grayscale image. Width ≈ SE size.

2.2 Top-hat (white top-hat)

```
top_hat = original − opening
```

Opening removes small bright features. Subtracting gives just those small bright features, on a near-black background. Great for finding bright spots, text on a dark background, or correcting uneven illumination.

2.3 Black-hat

```
black_hat = closing − original
```

Closing fills small dark features. Subtracting gives just those small dark features. Used for finding dark text on a bright background or dark dust spots.

2.4 OpenCV API

```
cv2.morphologyEx(img, cv2.MORPH_GRADIENT,   se)
cv2.morphologyEx(img, cv2.MORPH_TOPHAT,     se)
cv2.morphologyEx(img, cv2.MORPH_BLACKHAT,   se)
```

2.5 Use case: text extraction

For text on uneven-lit paper:

```
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
se   = cv2.getStructuringElement(cv2.MORPH_RECT, (25, 25))
# Dark text on bright paper → use black-hat
bh = cv2.morphologyEx(gray, cv2.MORPH_BLACKHAT, se)
_, text_mask = cv2.threshold(bh, 30, 255, cv2.THRESH_BINARY)
```

This isolates the text characters regardless of background brightness.

3. Code Examples

Example 1: Gradient — extract outlines

```
import cv2
from skimage.data import coins

_, binary = cv2.threshold(coins(), 80, 255, cv2.THRESH_BINARY)
se = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (3, 3))
grad = cv2.morphologyEx(binary, cv2.MORPH_GRADIENT, se)
cv2.imwrite("coins_outlines.png", grad)
```

Expected Output: Thin outlines around each coin.

Example 2: Top-hat — bright details on dark

```
import cv2
from skimage.data import camera

img = camera()
se = cv2.getStructuringElement(cv2.MORPH_RECT, (15, 15))
tophat = cv2.morphologyEx(img, cv2.MORPH_TOPHAT, se)
cv2.imwrite("camera_tophat.png", tophat)
```

Expected Output: Image where bright small features (highlights in the sky, white shirt buttons) stand out against a near-black background.

Example 3: Black-hat — dark text extraction

```
import cv2
from skimage.data import text

img = text()                    # rough page-of-text image
se = cv2.getStructuringElement(cv2.MORPH_RECT, (25, 25))
bh = cv2.morphologyEx(img, cv2.MORPH_BLACKHAT, se)
_, mask = cv2.threshold(bh, 30, 255, cv2.THRESH_BINARY)
cv2.imwrite("text_mask.png", mask)
```

Expected Output: Just the text glyphs, isolated cleanly on a black background, regardless of paper brightness gradient.

Example 4: Illumination correction with top-hat

```
import cv2, numpy as np
from skimage.data import camera

# Simulate uneven lighting: multiply with a smooth ramp
```

```
img = camera().astype(np.float32)
h, w = img.shape
ramp = np.tile(np.linspace(0.5, 1.5, w), (h, 1))
uneven = np.clip(img * ramp, 0, 255).astype(np.uint8)
cv2.imwrite("uneven.png", uneven)

se = cv2.getStructuringElement(cv2.MORPH_RECT, (51, 51))
bg = cv2.morphologyEx(uneven, cv2.MORPH_OPEN, se)   # background estimate
flat = cv2.subtract(uneven, bg)                      # remove background
cv2.imwrite("uneven_flat.png", flat)
```

Expected Output: uneven_flat.png has the brightness ramp removed — uniform background, foreground details preserved. This is the "rolling-ball" method, popular in microscopy.

4. Parameter Sensitivity

Param	Effect
SE size for gradient	thicker outlines
SE size for top-hat	larger = catches bigger features as "small"
SE size for black-hat	same logic for dark features

5. Common Mistakes

Mistake	What Happens	Fix
Wrong polarity (use top-hat when you needed black-hat)	Empty / inverted output	Decide: bright on dark or dark on bright
SE too small for top-hat	Output near zero	Use SE larger than the features you want to extract
Forgetting threshold after black-hat	Result is grayscale gradient, not binary	Threshold the output

6. Practice Tasks

- Compute morphological gradient of binarised coins().
- Apply top-hat (15×15) on camera().
- Use black-hat + threshold on text() to extract text.

7. Key Takeaways

- Gradient = dilation − erosion (boundaries).
- Top-hat = original − opening (bright small features).
- Black-hat = closing − original (dark small features).
- Top-hat / black-hat fix uneven lighting and isolate text / dust.

8. What's Next

Practical mask cleaning — putting the morphology toolbox to work in pipelines.

Lesson 5: Practical Mask Cleaning

Module: 10 — Morphological Operations

Duration: 25 minutes

Difficulty: Intermediate

Learning Objectives

- Pick a mask-cleaning pipeline based on defects observed.
- Separate touching objects with erosion.
- Reconnect broken edges with closing.

Prerequisites

- Module 10 Lesson 4

1. Why This Matters

In real pipelines you almost never threshold once and use the result. Masks need cleanup. Knowing which morphology op to reach for in each situation saves trial-and-error.

2. Concept

2.1 Defect → fix cheat-sheet

You see	Fix
Salt-and-pepper noise in mask	MORPH_OPEN with small SE
Holes inside white regions	MORPH_CLOSE
Touching objects you want separated	extra cv2.erode iterations
Broken object boundaries	MORPH_CLOSE with larger SE
Need just object outlines	MORPH_GRADIENT
Outlines too thick after gradient	smaller SE

2.2 Standard cleanup pipeline

```
def clean(mask, k=5):
    se = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (k, k))
    mask = cv2.morphologyEx(mask, cv2.MORPH_OPEN,  se)
    mask = cv2.morphologyEx(mask, cv2.MORPH_CLOSE, se)
    return mask
```

Adapt SE size to your image scale.

2.3 Separating touching objects

When two objects share a boundary, threshold treats them as one blob. Erode aggressively to "shrink" them into separate cores:

```
sure_fg = cv2.erode(mask, se, iterations=3)
```

The result is too small but each blob is now a distinct connected component. Use this with the distance transform / watershed (M11) for full separation.

2.4 Reconnecting broken edges

Pre-edge content with gaps can be sealed:

```
mask = cv2.morphologyEx(mask, cv2.MORPH_CLOSE,
                        cv2.getStructuringElement(cv2.MORPH_RECT, (15, 1)))   #
horizontal close
```

Asymmetric SE closes along one direction — great for sealing horizontal text lines.

2.5 Sanity check after cleaning

Count connected components before/after:

```
n, _ = cv2.connectedComponents(mask)
print("components:", n - 1)   # subtract 1 for background
```

A successful clean usually reduces noise components dramatically.

3. Code Examples

Example 1: Full clean pipeline

```
import cv2, numpy as np
from skimage.data import coins

raw_mask = cv2.threshold(coins(), 80, 255, cv2.THRESH_BINARY)[1]
# Add some synthetic noise
np.random.seed(0)
sp = np.random.rand(*raw_mask.shape)
raw_mask[sp < 0.01] = 0
raw_mask[sp > 0.99] = 255

se = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (5, 5))
opened = cv2.morphologyEx(raw_mask, cv2.MORPH_OPEN,  se)
cleaned = cv2.morphologyEx(opened,   cv2.MORPH_CLOSE, se)

combo = np.hstack([raw_mask, opened, cleaned])
cv2.imwrite("clean_pipeline.png", combo)
print("components before:", cv2.connectedComponents(raw_mask)[0] - 1)
print("components after :", cv2.connectedComponents(cleaned)[0] - 1)
```

Expected Output: 3 panels — noisy mask | speckles gone (open) | also holes filled (close). Component count drops dramatically.

Example 2: Aggressive erosion to separate touching objects

```
import cv2, numpy as np

# Two touching circles
mask = np.zeros((200, 400), dtype=np.uint8)
cv2.circle(mask, (150, 100), 80, 255, -1)
```

```
cv2.circle(mask, (250, 100), 80, 255, -1)

se = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (3, 3))
core = cv2.erode(mask, se, iterations=15)

n_raw,  _ = cv2.connectedComponents(mask)
n_core, _ = cv2.connectedComponents(core)
print("raw components:", n_raw - 1, "  core components:", n_core - 1)
cv2.imwrite("touching_raw.png", mask)
cv2.imwrite("touching_core.png", core)
```

Expected Output: raw=1 (one merged blob), core=2 (two separated). Eroded cores are smaller but distinct.

Example 3: Reconnect broken text lines

```
import cv2, numpy as np

# Synthesize broken horizontal lines
mask = np.zeros((100, 300), dtype=np.uint8)
mask[20:25,  10: 80] = 255
mask[20:25,  90:160] = 255      # gap at 80-90
mask[20:25, 170:240] = 255      # gap at 160-170

se = cv2.getStructuringElement(cv2.MORPH_RECT, (15, 1))
closed = cv2.morphologyEx(mask, cv2.MORPH_CLOSE, se)
cv2.imwrite("text_lines_broken.png", mask)
cv2.imwrite("text_lines_closed.png", closed)
```

Expected Output: The broken horizontal segments become a single continuous line.

Example 4: Connected components for diagnosis

```
import cv2
from skimage.data import coins

_, mask = cv2.threshold(coins(), 80, 255, cv2.THRESH_BINARY)
n, _ = cv2.connectedComponents(mask)
print("blobs in raw mask:", n - 1)

se = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (5, 5))
clean = cv2.morphologyEx(mask, cv2.MORPH_OPEN, se)
n2, _ = cv2.connectedComponents(clean)
print("blobs after open  :", n2 - 1)
```

Expected Output: Component count typically drops after opening — noise specks removed.

4. Parameter Sensitivity

Param	Effect
SE size	bigger removes more / merges more
Iterations	linear stacking
SE asymmetry	direction-specific cleaning

5. Common Mistakes

Mistake	What Happens	Fix
Skip cleanup; pipe noisy mask to contour stage	Hundreds of bogus contours	Open + close first
Over-erode to separate objects	All objects vanish	Use minimum erosion + watershed (M11)
Wrong SE direction	Won't reconnect gaps	Use a row / column SE for directional ops

6. Practice Tasks

- Make a noisy binarised coins(); apply the standard open-close pipeline.
- Build two touching circles; erode them apart and count components.
- Build broken horizontal segments; reconnect with a (15, 1) closing.

7. Key Takeaways

- Match the op to the defect (cheat-sheet).
- Standard cleanup = OPEN then CLOSE.
- Aggressive erosion separates touching objects.
- Asymmetric SEs handle direction-specific defects.

8. What's Next

End of Module 10. Next: Module 11 — Segmentation & Thresholding.

Module 10 — Exercises

15 exercises.

10.1 Binary & SEs

- (E) Build a 7×7 ELLIPSE SE; print.
- (M) Build a 9×9 CROSS SE; print.
- (H) Threshold coins() at 80; save mask.

10.2 Erode / Dilate

- (E) Erode binarised coins with 5×5 ellipse.
- (M) Dilate the same; compare.
- (H) Verify pure-NumPy erode against cv2.erode on a small 20×20 patch.

10.3 Open / Close

- (E) Build 15% speckle mask; apply open(5).
- (M) Build holed square; apply close(5).
- (H) Open-then-close on binarised coins() and count components before/after.

10.4 Gradient / Top-hat / Black-hat

- (E) Gradient of binarised coins.
- (M) Top-hat (15×15) on camera().
- (H) Black-hat + threshold on text().

10.5 Practical

- (E) Standard clean pipeline on noisy coins() mask.
- (M) Erode 2 touching circles apart; count components.
- (H) Reconnect broken horizontal segments via (15,1) close.

Module 10 — Quiz

10 MCQs.

Q1

Erosion on white pixels: A. grows them B. shrinks them C. inverts them D. no change

Answer: B (L2)

Q2

Dilation on white pixels: A. grows them B. shrinks them C. inverts them D. blurs

Answer: A (L2)

Q3

Opening removes: A. small white specks B. holes C. edges D. everything

Answer: A (L3)

Q4

Closing fills: A. small white specks B. small black holes C. edges D. nothing

Answer: B (L3)

Q5

Opening = ? A. dilate then erode B. erode then dilate C. dilate twice D. erode twice

Answer: B (L3)

Q6

Morphological gradient = ? A. dilate − erode B. erode − dilate C. open − close D. close − open

Answer: A (L4)

Q7

Top-hat extracts: A. dark small features B. bright small features C. edges D. holes

Answer: B (L4)

Q8

For black text on bright paper, you'd use: A. top-hat B. black-hat C. gradient D. dilation

Answer: B (L4)

Q9

For separating two touching objects, the right first step is: A. dilation B. opening C. erosion (multiple iterations) D. closing

Answer: C (L5)

Q10

Standard noisy-mask cleanup is: A. open + close B. erode 10x C. blur D. invert

Answer: A (L5)

Module 10 — Assignment: Mask Cleaner

Estimated time: 1.5 hours

Combines: Module 10 + Modules 4, 6, 9

Brief

CLI tool that takes a binary mask (or makes one via thresholding), reports its noise stats, and cleans it.

Requirements

- mask_clean.py INPUT [--threshold 128] [--ksize 5] [--output OUT]
- If input is color, convert to gray and threshold.
- Compute: connected components before clean.
- Apply opening + closing with ellipse SE.
- Report components after.
- Save cleaned mask.

Constraints

- Modules 1-10 only.
- Use cv2.connectedComponents.
- Use argparse.

Expected Output

```
$ python mask_clean.py coins.png --threshold 80
components before: 73
components after : 18
saved -> coins_mask_clean.png
```

Grading Rubric

Criterion	Weight
Reads + thresholds	20%
Component count	20%
Open + close pipeline	30%
Reporting	15%
Style	15%

Submission

Save as assignment_module10.py.

Module 10 — Mini Project: Morphology Playground

Overview

Interactive morphology playground. Load any image, threshold it, then experiment with erosion / dilation / opening / closing / gradient / top-hat / black-hat live.

Concepts Used

- All of Module 10
- Trackbars (M2 L3), threshold (M11 preview)

Features

- Trackbars: threshold, ksize, op-type (0..6), iterations.
- Side-by-side: thresholded mask | morphology result.
- Save (s), quit (q).

Starter Code

morph_playground.py:

```
import sys, cv2, numpy as np

OPS = ["erode", "dilate", "open", "close", "gradient", "tophat", "blackhat"]

def apply_op(mask, op, k, it):
    se = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (k, k))
    if op == "erode":     return cv2.erode(mask, se, iterations=it)
    if op == "dilate":    return cv2.dilate(mask, se, iterations=it)
    if op == "open":      return cv2.morphologyEx(mask, cv2.MORPH_OPEN,    se,
iterations=it)
    if op == "close":     return cv2.morphologyEx(mask, cv2.MORPH_CLOSE,   se,
iterations=it)
    if op == "gradient":  return cv2.morphologyEx(mask, cv2.MORPH_GRADIENT, se)
    if op == "tophat":    return cv2.morphologyEx(mask, cv2.MORPH_TOPHAT,   se)
    if op == "blackhat":  return cv2.morphologyEx(mask, cv2.MORPH_BLACKHAT, se)

def run(path):
    img = cv2.imread(path, cv2.IMREAD_GRAYSCALE)
    if img is None: raise FileNotFoundError(path)
    if max(img.shape) > 700: img = cv2.resize(img, None, fx=0.5, fy=0.5)

    cv2.namedWindow("morph")
    cv2.createTrackbar("threshold", "morph", 128, 255, lambda v: None)
    cv2.createTrackbar("ksize",     "morph",   5,  31, lambda v: None)
    cv2.createTrackbar("op",        "morph",   2,   6, lambda v: None)   # 0..6
    cv2.createTrackbar("iter",      "morph",   1,  10, lambda v: None)

    while True:
        th = cv2.getTrackbarPos("threshold", "morph")
        k  = max(1, cv2.getTrackbarPos("ksize", "morph"))
        if k % 2 == 0: k += 1
```

```
        op_idx = cv2.getTrackbarPos("op", "morph")
        it = max(1, cv2.getTrackbarPos("iter", "morph"))

        _, mask = cv2.threshold(img, th, 255, cv2.THRESH_BINARY)
        out = apply_op(mask, OPS[op_idx], k, it)
        combo = np.hstack([mask, out])
        cv2.putText(combo, f"{OPS[op_idx]} k={k} it={it} th={th}", (10, 20),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.5, 128, 1)
        cv2.imshow("morph", combo)
        kk = cv2.waitKey(30) & 0xFF
        if kk == ord('q'): break
        if kk == ord('s'): cv2.imwrite("morph_out.png", out); print("saved
morph_out.png")
    cv2.destroyAllWindows()

if __name__ == "__main__":
    run(sys.argv[1] if len(sys.argv) > 1 else "datasets/starter/coins.png")
```

Expected Interaction

Slide threshold to set a mask, pick op, tune ksize / iterations live.

Extension Challenges

- Add a connected-component count badge on the result.
- Add "show original" toggle (o).
- Add asymmetric SE option ((15,1) etc.).

Grading Rubric

Criterion	Weight
All 7 ops work	35%
Threshold + ksize live	25%
Side-by-side display	15%
Save / quit	10%
Style	15%